



Storitveno usmerjene arhitekture

izr. prof. dr. Boštjan Šumak

1

Nazadnje

- Contract-First razvoj storitev
- Tehnični vmesniki spletnih storitev
- Primer razvoja tehničnega vmesnika
- Razvoj storitev po pristopu ContractFirst (Java, .NET)

- 1. kolokvij

2

JavaScript in Contract-First razvoj spletnih storitev SOAP

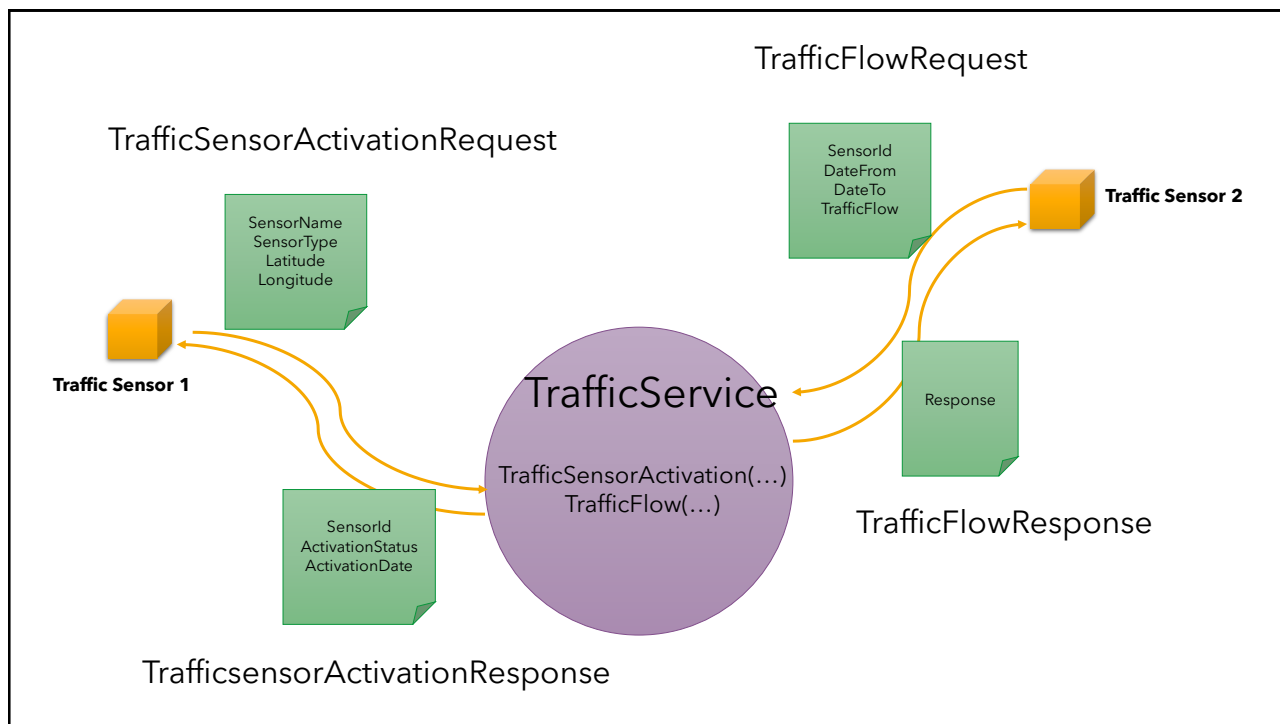
3

Contract-First: primer

- Zgraditi želimo storitev **TrafficService**, ki bo omogočala spremljanje pretoka prometa na različnih lokacijah (križiščih) v mestu.
 - Za potrebe meritev bodo nameščeni senzorji, ki bodo omogočali beleženje toka in pošiljanje podatkov o prometu na storitev.
- Operaciji, ki jih mora zagotoviti storitev TrafficService sta:
 - *TrafficSensorActivationResponse* **TrafficSensorActivation**(*TrafficSensorActivationRequest*) in
 - *TrafficFlowResponse* **TrafficFlow**(*TrafficFlow*)
- Sporočila naj vsebujejo na primer naslednje podatke:

Sporočilo	Podatki
TrafficSensorActivationRequest	SensorName, SensorType, Latitude, Longitude
TrafficSensorActivationResponse	SensorId, ActivationStatus, ActivationDate
TrafficFlowRequest	SensorId, DateFrom, DateTo, TrafficFlow
TrafficFlowResponse	Response

4



5

Razvoj SOAP storitev v JavaScript

- Za JavaScript zaenkrat nimamo orodij za razvoj po principu bottom-up (ali code-first)
- Prav tako ni orodij za razvoj po pristopu Contract-first
- V osnovi razvijamo po pristopu Contract-first
 - vmesnik spletne storitve moramo razviti sami (WSDL)
 - storitev moramo implementirati sami skladno z vmesnikom WSDL

6

```

<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<wsdl:definitions xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/" xmlns:tns="http://si.feri.soa/trafficService"
xmlns:data="http://si.feri.soa/trafficService/data" xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/" xmlns:ns1="http://schemas.xmlsoap.org/soap/http"
xmlns:xs="http://www.w3.org/2001/XMLSchema" name="TrafficService" targetNamespace="http://si.feri.soa/trafficService">

  <wsdl:types>
    <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" xmlns:tns="http://si.feri.soa/trafficService/data" elementFormDefault="unqualified"
      targetNamespace="http://si.feri.soa/trafficService/data">

      <xs:element name="TrafficSensorActivationRequest">
        <xs:complexType>
          <xs:sequence>
            <xs:element name="SensorName" type="xs:string"/>
            <xs:element name="SensorType" type="xs:string"/>
            <xs:element name="Latitude" type="xs:string"/>
            <xs:element name="Longitude" type="xs:string"/>
          </xs:sequence>
        </xs:complexType>
      </xs:element>

      <xs:element name="TrafficSensorActivationResponse">
        <xs:complexType>
          <xs:sequence>
            <xs:element name="SensorID" type="xs:long"/>
            <xs:element name="ActivationStatus" type="xs:string"/>
            <xs:element name="ActivationDate" type="xs:string"/>
          </xs:sequence>
        </xs:complexType>
      </xs:element>

      <xs:element name="TrafficFlowRequest">
        <xs:complexType>
          <xs:sequence>
            <xs:element name="SensorID" type="xs:long"/>
            <xs:element name="DateFrom" type="xs:string"/>
            <xs:element name="DateTo" type="xs:string"/>
            <xs:element name="TrafficFlow" type="xs:string"/>
          </xs:sequence>
        </xs:complexType>
      </xs:element>

      <xs:element name="TrafficFlowResponse">
        <xs:complexType>
          <xs:sequence>
            <xs:element name="response" type="xs:string"/>
          </xs:sequence>
        </xs:complexType>
      </xs:element>
    </xs:schema>
  </wsdl:types>

```

wsdl (modificiran)
1/2

7

```

<wsdl:message name="TrafficSensorActivationRequest">
  <wsdl:part name="params" element="data:TrafficSensorActivationRequest"/>
</wsdl:message>
<wsdl:message name="TrafficSensorActivationResponse">
  <wsdl:part name="params" element="data:TrafficSensorActivationResponse"/>
</wsdl:message>
<wsdl:message name="TrafficFlowRequest">
  <wsdl:part name="params" element="data:TrafficFlowRequest"/>
</wsdl:message>
<wsdl:message name="TrafficFlowResponse">
  <wsdl:part name="params" element="data:TrafficFlowResponse"/>
</wsdl:message>

<wsdl:portType name="TrafficService">
  <wsdl:operation name="TrafficSensorActivation">
    <wsdl:input message="tns:TrafficSensorActivationRequest"/>
    <wsdl:output message="tns:TrafficSensorActivationResponse"/>
  </wsdl:operation>
  <wsdl:operation name="TrafficFlow">
    <wsdl:input message="tns:TrafficFlowRequest"/>
    <wsdl:output message="tns:TrafficFlowResponse"/>
  </wsdl:operation>
</wsdl:portType>

<wsdl:binding name="TrafficServiceSOAP" type="tns:TrafficService">
  ...
</wsdl:binding>

<wsdl:service name="TrafficService">
  <wsdl:port binding="tns:TrafficServiceSOAP" name="TrafficServiceSOAP">
    <soap:address location="http://localhost:9090/TrafficService"/>
  </wsdl:port>
</wsdl:service>

</wsdl:definitions>

```

wsdl (modificiran)
2/2

8

trafficService.js (1/2)

```
const soap = require('soap');
const express = require('express');
const fs = require('fs');
const uuid = require('uuid');
const wsdl_file = fs.readFileSync("TrafficService.wsdl", "utf-8");

const TrafficSensorActivationImpl = function(params){
  const res = {
    SensorID: uuid.v4(),
    ActivationStatus: 'Activated',
    ActivationDate: '2.12.2025'
  };
  return res;
}

const TrafficFlowImpl = function(params){
  const res = {
    Response: 'OK'
  }
  return res;
}
```

9

trafficService.js (2/2)

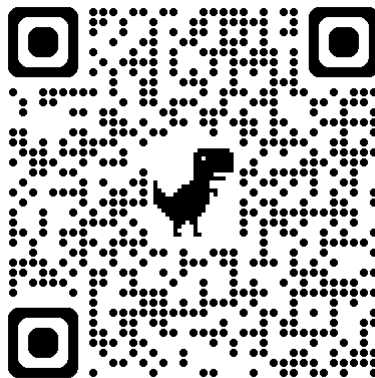
```
const serviceObject = {
  TrafficService: {
    TrafficServiceSOAP: {
      TrafficSensorActivation: TrafficSensorActivationImpl,
      TrafficFlow: TrafficFlowImpl
    }
  }
}

const path = "/TrafficService";
const app = express();

app.listen(9090, ()=>{
  soap.listen(app, path, serviceObject, wsdl_file, ()=>{
    console.log("Storitev je na voljo na http://localhost:9090/TrafficService?wsdl")
  });
});
```

10

Contract-First – ZAKAJ/KDAJ DA?



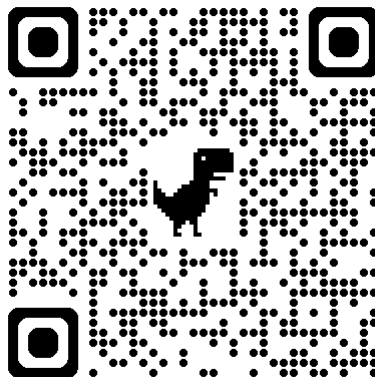
menti.com 4955 7328

11



12

Contract-First - ZAKAJ/KDAJ NE?



menti.com 4955 7328

13

A screenshot of a Menti poll interface. At the top right, there is a user profile icon with the initials 'BS' and a dropdown arrow. Below this, the text 'Menti' is displayed, followed by the poll title 'Contract-First razvoj sto...' and two icons: a share icon and a refresh icon. A section titled 'Choose a slide to present' contains two slide thumbnails. The first thumbnail is titled 'Contract-First - Zakaj DA?' and the second is titled 'Contract-First - Zakaj/Idaj ne?'. The second thumbnail is highlighted with a blue border. The interface is clean and modern, with a white background and blue accents.

14

Interoperabilnost spletnih storitev

15

Pomen interoperabilnosti

- Spletne storitve temeljijo na standardnih tehnologijah
 - XML, SOAP, WSDL, UDDI, ...
- Da bodo spletne storitve dejansko izkoristile potencial, morajo zagotavljati **visok nivo interoperabilnosti**

16

Kdaj je storitev interoperabilna?

- Ko
 - jo lahko uporabijo odjemalci, ki so implementirani s poljubnim programskim jezikom in tečejo na poljubni platformi
 - je sposobna komunicirati s katerokoli drugo spletno storitvijo

**Interoperabilnost zahteva jasno
razumevanje zahtev in
dosledno upoštevanje
specifikacij!**

17

WS-I

- Za potrebe zagotavljanja interoperabilnosti spletnih storitev SOAP je bila ustanovljena organizacija WS-I, katere cilji so bili predvsem
 - Zagotavljanje **izobraževanja in napotkov**, ki pripomorejo k večji sprejetosti in uporabi spletnih storitev
 - Promocija **konsistentnih in zanesljivih praks**, ki razvijalcem omogočajo razvoj interoperabilnih spletnih storitev
 - Promocija **skupne industrijske vizije za spletne storitve** – za sistematičen razvoj – evolucijo tehnologij spletnih storitev
- Za doseganje teh ciljev je WS-I izvajala več strategij, na primer:
 - **Navodila za implementacijo in testiranje** (najboljše prakse)
 - **Profile spletnih storitev** (razvoj testnih orodij)

18

Profili

- Nastali kot odziv na vedno večje število specifikacij, ki se uporabljajo skupaj za razvoj storitev
 - Različne verzije specifikacij
 - Posamezne specifikacije so se razvijale hitreje kot druge
 - Pogosto je prihajalo do konfliktov pri izpolnjevanju zahtev
- Vsebujejo seznam specifikacij tehnologij spletnih storitev (ime + verzija specifikacije) skupaj z navodili za implementacijo
 - kako je potrebno uporabljati posamezne komponente skupaj za razvoj interoperabilnih storitev
 - kjer profil ne definira eksplicitnih navodil/omejitve, veljajo pravila/omejitve iz specifikacije, ki je vključena

19

WS-I Basic Profile

- Osnovni profil, ki vsebuje naslednje komponente:
 - **Sporočanje (SOAP/HTTP)** - izmenjava sporočil čez omrežje, kar mogoča komunikacijo med vmesniki spletnih storitev
 - **Opis (WSDL)** - mehanizem skozi katerega se izpostavijo definicije/vmesniki spletnih storitev. Definirana so pravila, ki jih morajo upoštevati razvijalci spletnih storitev pri pošiljanju SOAP sporočil
 - **Odkrivanje (UDDI)** - metapodatki, ki omogočajo objavo spletnih storitev
 - **XML Schema** - način strukturiranja podatkov v SOAP sporočilu
 - **XML 1.0** - način serializacije (zapisa) XML podatkov pred prenosom čez žico (ali datoteko).

20

WS-I Basic Profile

- WS-I Basic Profile 1.0
 - <http://www.ws-i.org/profiles/BasicProfile-1.0-2004-04-16.html>
- WS-I Basic Profile 1.2
 - <http://ws-i.org/Profiles/BasicProfile-1.2-2010-11-09.html>
- WS-I Basic Profile 2.0
 - <http://ws-i.org/Profiles/BasicProfile-2.0-2010-11-09.html>

21

WS-I Basic Profile 1.0 (na primer)

- SOAP 1.1
- WSDL 1.1
- UDDI 2.0
- XML 1.0 (Second Edition)
- XML Schema Part 1: Structures
- XML Schema Part 2: Datatypes
- RFC2246: The Transport Layer Security Protocol Version 1.0
- RFC2459: Internet X.509 Public Key Infrastructure Certificate in CRL Profile
- RFC2616: HyperText Transfer Protocol 1.1
- RFC2818: HTTP over TLS
- RFC2965: HTTP State Management Mechanism
- The Secure Sockets Layer Protocol Version 3.0

22

Kaj/zakaj pomembno za razvijalce spletnih storitev?

- Specifikacije WS-I Basic Profile 1.0 so precej kompleksne
 - Večina specifikacij WS-I Basic Profile se ukvarja z razvojnimi orodji, SOAP procesorji, WSDL parserji, generatorji kode
- Razvijalci spletnih storitev morajo razumeti predvsem naslednja področja specifikacij WS-I Basic Profile:
 - **Poglavje 4:** nanaša se na SOAP in uporabo HTTP Bindinga (pomembno predvsem za razvijalce SOAP procesorjev)
 - **Poglavje 5:** nanaša se na pravila definiranja WSDL vmesnikov (pomembno predvsem na tiste, ki pišejo vmesnike spletnih storitev)
 - **Poglavje 6:** nanaša se na odkrivanje spletnih storitev in UDDI (pomembno za razvijalce spletnih storitev)
 - **Poglavje 7:** nanaša se na varnost spletnih storitev z uporabo HTTP/S (pomembno predvsem za razvijalce WS, ki razvijajo varne storitve)

23

Pisanje WSDL, ki je skladen z WS-I Basic Profile

- Poglavje 5 (<http://www.ws-i.org/profiles/BasicProfile-1.0-2004-04-16.html#description>)
 - navodila, ki jih je potrebno upoštevati pri pisanju WSDL
- WS-I Basic Profile **prepoveduje uporabo SOAP kodiranja vsebine sporočil** (vsebine v elementu body), ker se je v preteklosti pogosto izkazalo, da je povzročalo težave z interoperabilnostjo
- WS-I Basic Profile zahteva uporabo SOAP Bindinga v naslednjih kombinacijah
 - RPC/literal
 - document/literal

R2705 A wsdl:binding in a DESCRIPTION MUST use either be a rpc-literal binding or a document-literal binding.

R2706 A wsdl:binding in a DESCRIPTION MUST use the value of "literal" for the use attribute in all soapbind:body, soapbind:fault, soapbind:header and soapbind:headerfault elements.

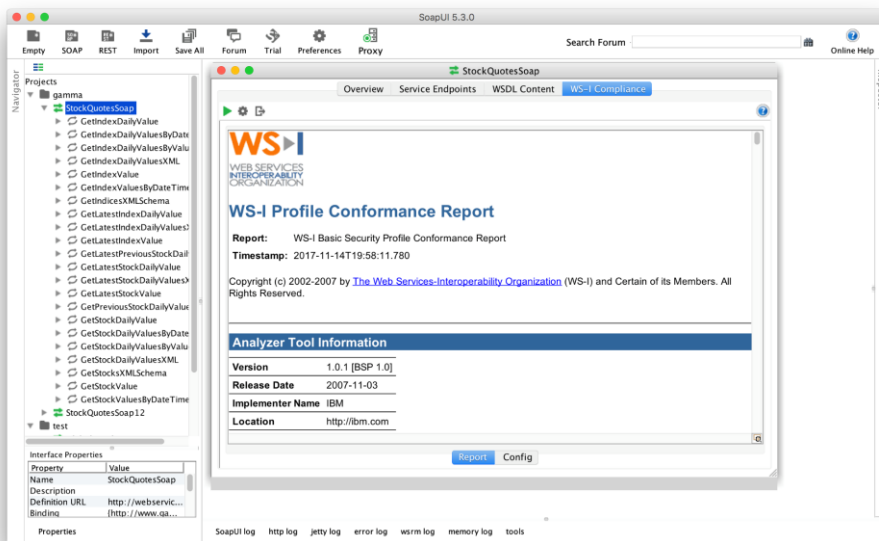
24

Orodja za validacijo vmesnikov spletnih storitev

- Razvojna orodja
- SoapUI

25

SoapUI



26

Kaj potrebujemo za razvoj **odjemalca spletne storitve** **SOAP?**

27

Izgradnja odjemalca storitve SOAP

- Kaj pomeni proženje storitve SOAP?
- Kaj lahko uporabimo, če razvijamo s programskim jezikom
 - Java
 - C#
 - JavaScript
 - ...

28

Razvoj odjemalcev SOAP v prog. jezikih in platformah

- V različnih programskih jezikih in platformah imamo različno podporo za proženje in uporabo storitev SOAP
- Skozi razrede in vmesnike, ki se avtomatsko zgradijo na osnovi WSDL je **abstrahirana logika**, ki zagotovi
 - **Oblikovanje zahteve SOAP**
 - Serializacija parametrov oz. objektov v sporočilo XML, ki se vključi v SOAP ovojnico
 - **Pošiljanje zahteve SOAP do ponudnika storitve**
 - Uporaba ustreznih protokolov za prenos (HTTP/HTTPS, ...)
 - **Procesiranje odgovora SOAP**
 - Deserializacija vsebine XML iz ovojnice SOAP in posredovanje podatkov v obliki objektov

29

Odjemalci spletnih storitev v programskem jeziku Java

30

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
<groupId>si.feri.soa.davcnablagajna</groupId>
<artifactId>davcna-blagajna-client</artifactId>
<version>1.0-SNAPSHOT</version>
```

```
<properties>
  <maven.compiler.release>21</maven.compiler.release>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <metro.version>3.0.2</metro.version>
  <davcna.wsdl.url>http://localhost:8081/davcnablagajna?wsdl</davcna.wsdl.url>
</properties>
```

```
<dependencies>
  <dependency>
    <groupId>com.sun.xml.ws</groupId>
    <artifactId>jaxws-maven-plugin</artifactId>
    <version>${metro.version}</version>
  </dependency>
</dependencies>
```

```
</project>
```

```
<build>
  <plugins>
    <plugin>
      <groupId>com.sun.xml.ws</groupId>
      <artifactId>jaxws-maven-plugin</artifactId>
      <version>${metro.version}</version>
      <executions>
        <execution>
          <id>generate-davcna-sync</id>
          <goals>
            <goal>wsimport</goal>
          </goals>
          <phase>generate-sources</phase>
        </execution>
      </executions>
      <configuration>
        <wsdlUrls>
          <wsdlUrl>${davcna.wsdl.url}</wsdlUrl>
        </wsdlUrls>
        <packageName>
          si.feri.soa.davcnablagajna.client
        </packageName>
        <bindingDirectory>
          ${project.basedir}/src/jaxws-sync
        </bindingDirectory>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Uporabimo jaxws-maven-plugin, ki ovije orodje **wsimport**.

Pove mu, kje je WSDL in v kateri paket naj generira odjemalca.

32

mvn clean generate-sources: sinhroni vmesnik

```
...
public interface DavcnaBlagajna {
    @WebMethod(operationName = "NovRacun")
    @WebResult(targetNamespace = "")
    @RequestWrapper(localName = "NovRacun", targetNamespace = "http://si.feri.soa/davcnablagajna/service/1.0", className =
        "si.feri.soa.davcnablagajna.client.sync.NovRacun")
    @ResponseWrapper(localName = "NovRacunResponse", targetNamespace = "http://si.feri.soa/davcnablagajna/service/1.0", className =
        "si.feri.soa.davcnablagajna.client.sync.NovRacunResponse")
    @Action(input = "http://si.feri.soa/davcnablagajna/service/1.0/DavcnaBlagajna/NovRacunRequest", output =
        "http://si.feri.soa/davcnablagajna/service/1.0/DavcnaBlagajna/NovRacunResponse")

    public Racun novRacun( @WebParam(name = "racun", targetNamespace = "") Racun racun);

    @WebMethod
    @WebResult(targetNamespace = "")
    @RequestWrapper(localName = "novPoslovniProstor", targetNamespace = "http://si.feri.soa/davcnablagajna/service/1.0", className =
        "si.feri.soa.davcnablagajna.client.sync.NovPoslovniProstor")
    @ResponseWrapper(localName = "novPoslovniProstorResponse", targetNamespace = "http://si.feri.soa/davcnablagajna/service/1.0", className =
        "si.feri.soa.davcnablagajna.client.sync.NovPoslovniProstorResponse")
    @Action(input = "http://si.feri.soa/davcnablagajna/service/1.0/DavcnaBlagajna/novPoslovniProstorRequest", output =
        "http://si.feri.soa/davcnablagajna/service/1.0/DavcnaBlagajna/novPoslovniProstorResponse")

    public PoslovniProstor novPoslovniProstor(@WebParam(name = "poslovniProstor", targetNamespace = "")
        PoslovniProstor poslovniProstor);
}
```

33

Sinhroni odjemalec

- V generiranem vmesniku DavcnaBlagajna vidimo dve operaciji:
 - novRacun in
 - novPoslovniProstor
- Obe sta sinhroni, vrneta direktno Racun oziroma PoslovniProstor

34

Ustvarjanje odjemalca v Javi

- Ustvarimo service in port iz generirane kode
- Pripravimo objekt Racun
- Kličemo sinhrono metodo novRacun(...)
- Nit se blokira, dokler rezultat ni znan

35

Primer – sinhroni odjemalec

```
package si.feri.soa.davcnablagajna;
import si.feri.soa.davcnablagajna.client.DavcnaBlagajna;
import si.feri.soa.davcnablagajna.client.DavcnaBlagajnaServiceService;
import si.feri.soa.davcnablagajna.client.Racun;

public class ClientSync {
    public static void main(String[] args) {
        // 1. Service (proxy do SOAP storitve)
        DavcnaBlagajnaServiceService service = new DavcnaBlagajnaServiceService();
        // 2. Port (vmesnik z operacijami)
        DavcnaBlagajna port = service.getDavcnaBlagajnaServicePort();
        // 3. Pripravimo primer računa
        Racun racun = new Racun();
        racun.setDavcnaSt("12345678");
        racun.setDatum("2025-12-01");
        racun.setZnesek(100.0);
        racun.setPpId("POS01");
        // 4. Sinhron klic – blokira do odgovora
        System.out.println("Prošim operacijo novRacun");
        Racun rezultat = port.novRacun(racun);
        System.out.println("Ustvarjen račun ID: " + rezultat.getId());
    }
}
```

Tipičen sinhroni odjemalec: najprej ustvarimo objekt tipa *Service*, na njem dobimo objekt tipa *Port*, pripravimo podatke in pokličemo metodo → glavna nit čaka, dokler strežnik ne vrne odgovora.

36

Kaj potrebujemo za asinhrono proženje storitve?

- Vmesnik, ki ponuja asinhrono operacije

37

Asinhrono proženje metod spletne storitve v Javi

- V projektu odjemalca v mapo `src/jaxws/` dodamo na primer datoteko:

`src/jaxws/async-bindings.xml`

```
<jaxws:bindings
  xmlns:jaxws="https://jakarta.ee/xml/ns/jaxws"
  xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
  wsdlLocation="http://localhost:8081/davcnablagajna?wsdl"
  version="3.0">

  <jaxws:bindings node="wsdl:definitions">
    <jaxws:enableAsyncMapping>true</jaxws:enableAsyncMapping>
  </jaxws:bindings>

</jaxws:bindings>
```

38

Pom.xml – sprememba v kodi za generiranje

```
<execution>
  <id>generate-davcna-async</id>
  <goals>
    <goal>wsimport</goal>
  </goals>
  <phase>generate-sources</phase>
  <configuration>
    <wsdlUrls>
      <wsdlUrl>${davcna.wsdl.url}</wsdlUrl>
    </wsdlUrls>
    <packageName>
      si.feri.soa.davcnablagajna.client
    </packageName>
    <bindingFiles>
      <bindingFile>async-bindings.xml</bindingFile>
    </bindingFiles>
  </configuration>
</execution>
```

39

mvn clean generate-sources: asinhroni vmesnik (1/2)

```
@WebMethod(operationName = "NovRacun")
@RequestWrapper(localName = "NovRacun", targetNamespace = "http://si.feri.soa/davcnablagajna/service/1.0", className = "si.feri.soa.davcnablagajna.client.NovRacun")
@ResponseWrapper(localName = "NovRacunResponse", targetNamespace = "http://si.feri.soa/davcnablagajna/service/1.0", className = "si.feri.soa.davcnablagajna.client.NovRacunResponse")

public Response<NovRacunResponse> novRacunAsync(
    @WebParam(name = "racun", targetNamespace = "") Racun racun);

@WebMethod(operationName = "NovRacun")
@RequestWrapper(localName = "NovRacun", targetNamespace = "http://si.feri.soa/davcnablagajna/service/1.0", className = "si.feri.soa.davcnablagajna.client.NovRacun")
@ResponseWrapper(localName = "NovRacunResponse", targetNamespace = "http://si.feri.soa/davcnablagajna/service/1.0", className = "si.feri.soa.davcnablagajna.client.NovRacunResponse")

public Future<?> novRacunAsync(@WebParam(name = "racun", targetNamespace = "")Racun racun,
    @WebParam(name = "asyncHandler", targetNamespace = "")
    AsyncHandler<NovRacunResponse> asyncHandler);

@WebMethod(operationName = "NovRacun")
@WebResult(targetNamespace = "")
@RequestWrapper(localName = "NovRacun", targetNamespace = "http://si.feri.soa/davcnablagajna/service/1.0", className = "si.feri.soa.davcnablagajna.client.NovRacun")
@ResponseWrapper(localName = "NovRacunResponse", targetNamespace = "http://si.feri.soa/davcnablagajna/service/1.0", className = "si.feri.soa.davcnablagajna.client.NovRacunResponse")
@Action(input = "http://si.feri.soa/davcnablagajna/service/1.0/DavcnaBlagajna/NovRacunRequest", output = "http://si.feri.soa/davcnablagajna/service/1.0/DavcnaBlagajna/NovRacunResponse")

public Racun novRacun( @WebParam(name = "racun", targetNamespace = "") Racun racun);
```

40

mvn clean generate-sources: asinhroni vmesnik (2/2)

```
@WebMethod(operationName = "novPoslovniProstor")
@RequestWrapper(localName = "novPoslovniProstor", targetNamespace = "http://si.feri.soa/davcnablagajna/service/1.0", className = "si.feri.soa.davcnablagajna.client.NovPoslovniProstor")
@ResponseWrapper(localName = "novPoslovniProstorResponse", targetNamespace = "http://si.feri.soa/davcnablagajna/service/1.0", className = "si.feri.soa.davcnablagajna.client.NovPoslovniProstorResponse")

public Response<NovPoslovniProstorResponse> novPoslovniProstorAsync(
    @WebParam(name = "poslovniProstor", targetNamespace = "") PoslovniProstor poslovniProstor);

@WebMethod(operationName = "novPoslovniProstor")
@RequestWrapper(localName = "novPoslovniProstor", targetNamespace = "http://si.feri.soa/davcnablagajna/service/1.0", className = "si.feri.soa.davcnablagajna.client.NovPoslovniProstor")
@ResponseWrapper(localName = "novPoslovniProstorResponse", targetNamespace = "http://si.feri.soa/davcnablagajna/service/1.0", className = "si.feri.soa.davcnablagajna.client.NovPoslovniProstorResponse")

public Future<?> novPoslovniProstorAsync(
    @WebParam(name = "poslovniProstor", targetNamespace = "") PoslovniProstor poslovniProstor,
    @WebParam(name = "asyncHandler", targetNamespace = "") AsyncHandler<NovPoslovniProstorResponse> asyncHandler);

@WebMethod
@WebResult(targetNamespace = "")
@RequestWrapper(localName = "novPoslovniProstor", targetNamespace = "http://si.feri.soa/davcnablagajna/service/1.0", className = "si.feri.soa.davcnablagajna.client.NovPoslovniProstor")
@ResponseWrapper(localName = "novPoslovniProstorResponse", targetNamespace = "http://si.feri.soa/davcnablagajna/service/1.0", className = "si.feri.soa.davcnablagajna.client.NovPoslovniProstorResponse")
@Action(input = "http://si.feri.soa/davcnablagajna/service/1.0/DavcnaBlagajna/novPoslovniProstorRequest", output = "http://si.feri.soa/davcnablagajna/service/1.0/DavcnaBlagajna/novPoslovniProstorResponse")

public PoslovniProstor novPoslovniProstor(
    @WebParam(name = "poslovniProstor", targetNamespace = "") PoslovniProstor poslovniProstor);
```

41

Generiranje kode - asinhroni odjemalec

- Nismo zamenjali WSDL, nismo spreminjali storitve.
- Samo pri generiranju odjemalca smo vklopili dodatno generiranje asinhronih metod.
- V vmesniku je po generiranju šest metod:
 - Racun **novRacun**(Racun racun);
 - PoslovniProstor **novPoslovniProstor**(PoslovniProstor poslovniProstor);
 - Response<NovRacunResponse> **novRacunAsync**(Racun racun);
 - Future<?> **novRacunAsync**(Racun racun, AsyncHandler<NovRacunResponse> asyncHandler);
 - Response<NovPoslovniProstorResponse> **novPoslovniProstorAsync**(PoslovniProstor poslovniProstor);
 - Future<?> **novPoslovniProstorAsync**(PoslovniProstor poslovniProstor, AsyncHandler<NovPoslovniProstorResponse> asyncHandler);

42

Asinhrono proženje metod spletne storitve v Javi

- Za vsako operacijo imamo zdaj tri načine proženja:
 - sinhrono,
 - asinhrono z Response<T> (**polling**),
 - asinhrono z Future<?> in AsyncHandler<T> (**callback**).

43

Asinhroni odjemalec - Async Polling

- Metoda ne vrne direktno Racun, ampak Response<NovRacunResponse>.
- Rezultat pride kasneje, zato periodično sprašujemo isDone().
- Ko je rezultat pripravljen, ga preberemo z get().

44

Asinhroni odjemalec - Async Polling

```
package si.feri.soa.davcnablagajna;
import java.util.concurrent.TimeUnit;
import ...

public class ClieAsyncPolling {
    public static void main(String[] args) throws Exception {
        DavcnaBlagajnaService service = new DavcnaBlagajnaService();
        DavcnaBlagajna port = service.getDavcnaBlagajnaServicePort();
        Racun racun = new Racun();
        racun.setDavcnaSt("12345678");
        racun.setDatum("2025-12-01");
        racun.setZnesek(100.0);
        racun.setPpId("POS01");

        Response<NovRacunResponse> odg = port.novRacunAsync(racun);
        System.out.println("Čakam na rezultat...");

        while(!odg.isDone()){ //polling
            System.out.print(".");
            TimeUnit.MILLISECONDS.sleep(100);
        }
        System.out.println();
        System.out.println("Račun je bil registriran, id = " + odg.get().getReturn().getId());
    }
}
```

45

Asinhroni odjemalec - Async Callback

- Pri callback vzorcu nam ni treba ročno preverjati, ali je rezultat že prišel - registriramo AsyncHandler.
- Ko strežnik odgovori, framework pokliče handleResponse in tam obdelamo rezultat.
- Hkrati dobimo še Future, če bi kdaj želeli vseeno počakati na rezultat.

46

Asinhroni odjemalec - Async Callback

```
package si.feri.soa.davcnablagajna;
import java.util.concurrent.ExecutionException;
import ...
public class ClientAsyncCallback {
    public static void main(String[] args) throws InterruptedException, ExecutionException {
        DavcnaBlagajnaServiceService service = new DavcnaBlagajnaServiceService();
        DavcnaBlagajna port = service.getDavcnaBlagajnaServicePort();
        Racun racun = new Racun();
        racun.setDavcnaSt("12345678");
        racun.setDatum("2025-12-01");
        racun.setZnesek(100.0);
        racun.setPpld("POS01");
        // handler se izvede, ko rezultat prispe
        AsyncHandler<NovRacunResponse> handler = new AsyncHandler<>() {
            @Override
            public void handleResponse(Response<NovRacunResponse> response) {
                try {
                    NovRacunResponse odgovor = response.get();
                    Racun rez = odgovor.getReturn();
                    System.out.println("Račun registriran, id= " + rez.getId());
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        };
        // asinhroni klic s callback
        Future<?> future = port.newRacunAsync(racun, handler);
        System.out.println("Prožil metodo novRacunAsync. Odjemalec lahko medtem dela kaj drugega...");
        // tukaj blokiramo, dokler callback ne konča
        future.get(); // počaka, dokler asinhroni klic ni zaključen
    }
}
```

47

Odjemalci spletnih storitev v .NET

48

Iz Java v .NET

Java

- JAX-WS (**wsimport** preko **jaxws-maven-plugin**)
- **sinhrone** + **async metode** (Response/Future/AsyncHandler)
- V Javi smo se igrali s tem, ali bomo generirali sync ali async vmesnik

.NET (moderen, .NET 8+)

- **dotnet-svcutil** (WCF ServiceModel client generator)
- generira **async API**: Task<T> + async / await
- V modernem .NET svetu generator privzeto daje *asinhroni API* z Task<T>.
 - Skladno s Task-based Asynchronous Pattern (TAP) in z async / await.

49

Orodje dotnet-svcutil

dotnet tool install --global dotnet-svcutil

- Če je orodje že nameščeno, lahko ta korak preskočimo
- dotnet-svcutil je za .NET nekaj podobnega kot wsimport za Javo – iz WSDL generira odjemalski proxy.

51

Generiranje kode odjemalca

**dotnet-svcutil http://localhost:8081/davcnablagajna?wsdl **
--namespace "*,DavcnaBlagajnaOdjemalec.Proxy"

- Ukaz
 - ustvari datoteko (običajno) Reference.cs,
 - v njej bo razred npr. DavcnaBlagajnaClient ali podobno (ime vzame iz WSDL),
 - vključi tudi Racun, PoslovniProstor in ostale tipe.

52

Vmesnik storitve

```
[System.CodeDom.Compiler.GeneratedCodeAttribute("Microsoft.Tools.ServiceModel.Svcutil", "8.0.0")]
[System.ServiceModel.ServiceContractAttribute(Namespace="http://si.feri.soa/davcnablajajna/service/1.0", ConfigurationName="DavcnaBlagajnaOdjemalec.Proxy.DavcnaBlagajna")]
public interface DavcnaBlagajna
{
    [System.ServiceModel.OperationContractAttribute(Action="http://si.feri.soa/davcnablajajna/service/1.0/DavcnaBlagajna/NovRacunRequest",
    ReplyAction="http://si.feri.soa/davcnablajajna/service/1.0/DavcnaBlagajna/NovRacunResponse")]
    [System.ServiceModel.XmlSerializerFormatAttribute(SupportFaults=true)]
    System.Threading.Tasks.Task<DavcnaBlagajnaOdjemalec.Proxy.NovRacunResponse>
    NovRacunAsync(DavcnaBlagajnaOdjemalec.Proxy.NovRacunRequest request);

    [System.ServiceModel.OperationContractAttribute(Action="http://si.feri.soa/davcnablajajna/service/1.0/DavcnaBlagajna/novPoslovniProstorRequest",
    ReplyAction="http://si.feri.soa/davcnablajajna/service/1.0/DavcnaBlagajna/novPoslovniProstorResponse")]
    [System.ServiceModel.XmlSerializerFormatAttribute(SupportFaults=true)]
    System.Threading.Tasks.Task<DavcnaBlagajnaOdjemalec.Proxy.novPoslovniProstorResponse>
    novPoslovniProstorAsync(DavcnaBlagajnaOdjemalec.Proxy.novPoslovniProstorRequest request);
}
```

Enakovredno JAX-WS vmesniku v Javi (SEI) - vsaka operacija ima asinhrono metodo, ki vrača Task<...>:

- Task<NovRacunResponse>
- Task<novPoslovniProstorResponse>

53

Proxy razred: DavcnaBlagajnaClient

```
[System.Diagnostics.DebuggerStepThroughAttribute()]
[System.CodeDom.Compiler.GeneratedCodeAttribute("Microsoft.Tools.ServiceModel.Svcutil", "8.0.0")]
public partial class DavcnaBlagajnaClient : System.ServiceModel.ClientBase<DavcnaBlagajnaOdjemalec.Proxy.DavcnaBlagajna>, DavcnaBlagajnaOdjemalec.Proxy.DavcnaBlagajna
{
    ...
    [System.ComponentModel.EditorBrowsableAttribute(System.ComponentModel.EditorBrowsableState.Advanced)]
    System.Threading.Tasks.Task<DavcnaBlagajnaOdjemalec.Proxy.NovRacunResponse>
    DavcnaBlagajnaOdjemalec.Proxy.DavcnaBlagajna.NovRacunAsync(DavcnaBlagajnaOdjemalec.Proxy.NovRacunRequest request)
    {
        return base.Channel.NovRacunAsync(request);
    }

    public System.Threading.Tasks.Task<DavcnaBlagajnaOdjemalec.Proxy.NovRacunResponse> NovRacunAsync(DavcnaBlagajnaOdjemalec.Proxy.Racun racun)
    {
        DavcnaBlagajnaOdjemalec.Proxy.NovRacunRequest inValue = new DavcnaBlagajnaOdjemalec.Proxy.NovRacunRequest();
        inValue.racun = racun;
        return ((DavcnaBlagajnaOdjemalec.Proxy.DavcnaBlagajna)(this)).NovRacunAsync(inValue);
    }
    ...
}
```

DavcnaBlagajnaClient ima metode:

Task<NovRacunResponse> **NovRacunAsync**(Racun racun)
 Task<novPoslovniProstorResponse> **novPoslovniProstorAsync**(PoslovniProstor poslovniProstor),

54

.NET Odjemalec

```
using System;
using System.Threading.Tasks;
using DavcnaBlagajnaOdjemalec.Proxy;
namespace DavcnaBlagajnaOdjemalec
{
    internal class Program
    {
        static async Task Main(string[] args)
        {
            try {
                var client = new DavcnaBlagajnaClient();
                var racun = new Racun
                {
                    davcnaSt = "12345678",
                    datum = "2025-12-01",
                    znesek = 100.0,
                    ppId = "POS01"
                };
                Console.WriteLine("Pošiljam asinhrono zahtevo za NovRacun...");
                NovRacunResponse response = await client.NovRacunAsync(racun);
                Racun rezultat = response.@return;
                Console.WriteLine("ASYNC .NET – NovRacun");
                Console.WriteLine($"ID: {rezultat.id}");
                Console.WriteLine($"Davčna: {rezultat.davcnaSt}");
                Console.WriteLine($"Datum: {rezultat.datum}");
                Console.WriteLine($"Znesek: {rezultat.znesek}");
                Console.WriteLine($"PPID: {rezultat.ppId}");
            }
            catch (Exception ex) {
                Console.WriteLine($"Napaka pri klicu storitve: {ex.Message}");
            }
        }
    }
}
```

55

Odjemalci spletnih storitev v programskem jeziku JavaScript

56

JavaScript + SOAP

- SOAP = XML preko HTTP
- Java/.NET:
 - generatorji (wsimport, dotnet-svcutil) → močno tipizirani odjemalci.
- JavaScript:
 - ročno sestavimo SOAP envelope in pošljemo zahtevo s **fetch**
 - uporabimo Node.js modul (npr. soap), ki zna prebrati WSDL in nam da JS metode.
- V JavaScript svetu nimamo 'out-of-the-box' generatorjev,
 - zato se ali igramo na nizkem nivoju (HTTP + XML) ali
 - uporabimo Node.js modul

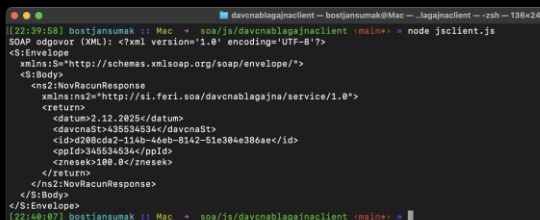
57

Preprosti Javascript odjemalec (fetch)

```
import { pd } from 'pretty-data';
const url = "http://localhost:8081/davcnablajajna";
const soapEnvelope = `
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:ns="http://si.feri.soa/davcnablajajna/service/1.0">
  <soapenv:Header/>
  <soapenv:Body>
    <ns:NovRacun>
      <racun>
        <datum>2.12.2025</datum>
        <davcnaSt>435534534</davcnaSt>
        <ppld>345534534</ppld>
        <znesek>100</znesek>
      </racun>
    </ns:NovRacun>
  </soapenv:Body>
</soapenv:Envelope>
`;

const response = await fetch(url, {
  method: "POST",
  headers: {
    "Content-Type": "text/xml;charset=utf-8"
  },
  body: soapEnvelope
});

const text = await response.text();
console.log("SOAP odgovor (XML): " + pd.xml(text));
```



```

[22:39:58] bostjansumak :: Mac - soa/js/davcnablajajnaclient -bostjansumak@Mac -- .lagajnaclient -- zsh -- 130x24
SOAP odgovor (XML): <?xml version="1.0" encoding="UTF-8"?>
<S:Envelope
  xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
  <S:Body>
    <ns2:NovRacunResponse
      xmlns:ns2="http://si.feri.soa/davcnablajajna/service/1.0">
      <return>
        <datum>2.12.2025</datum>
        <davcnaSt>435534534</davcnaSt>
        <id>2880cd2-11b-44eb-8142-51e304e386ae</id>
        <ppld>345534534</ppld>
        <znesek>100.0</znesek>
      </return>
      </ns2:NovRacunResponse>
    </S:Body>
  </S:Envelope>
[22:40:07] bostjansumak :: Mac - soa/js/davcnablajajnaclient -bostjansumak@Mac -- .lagajnaclient -- zsh -- 130x24

```

58

Odjemalec SOAP: Node.js + modul

- soap
- loopback
- easy-soap-request
- loopback-connector-soap
- fuel-soap
- @briggysbro/s SOAP
- soap-as-promised
- node-wsdl
- ...

59

Odjemalec SOAP z modulom **soap**

- <https://www.npmjs.com/package/soap>)
- Omogoča povezovanje s storitvami SOAP
- Omogoča izgradnjo in nudenje storitev SOAP
- Enostaven API
- Omogoča RPC in dokumentno usmerjeno sporočanje
- Podpora za sinhrono in asinhrono proženje
- Podpora za WS-Security

npm install soap

60

soap: primer asinhronnega odjemalca (**callback**)

```
const soap = require('soap');
const wsdl = "http://localhost:8081/davcnablagajna?wsdl";
const args = {
  racun : {
    datum: '2.12.2025',
    davcnaSt: '7188181919',
    ppId: '54454545',
    znesek: 100
  }
};

soap.createClient(wsdl, function(err, client){
  client.NovRacun(args, function(err, odg){
    console.log("Račun je bil registriran. Id= " + odg.return.id);
  });
});
```

61

soap: primer asinhronnega odjemalca (**await**)

```
import soap from 'soap';

const wsdl = 'http://localhost:8081/davcnablagajna?wsdl';
const client = await soap.createClientAsync(wsdl);
const args = {
  racun: {
    datum: '2025-12-02',
    davcnaSt: '7188181919',
    ppId: '54454545',
    znesek: 100
  }
};

console.log('Pošiljam SOAP zahtevo NovRacun (sinhroni flow z await)...');

const [result, rawResponse, soapHeader, rawRequest] =
  await client.NovRacunAsync(args);

console.log('Raw result object:');
console.log(JSON.stringify(result, null, 2));
```

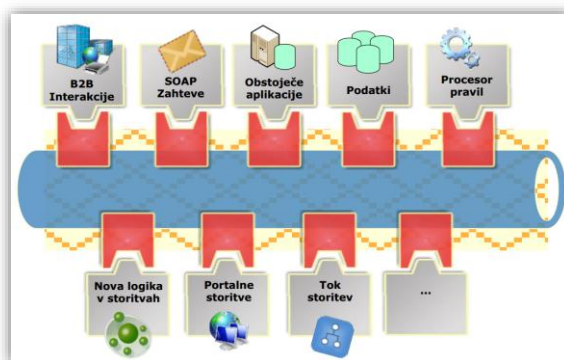
62

Storitveno vodilo

63

Storitveno vodilo

- Enterprise Service Bus (ESB)
- Vmesni sloj med poslovnimi aplikacijami
- Zagotavlja interoperabilnost med aplikacijami, ki uporabljajo različne komunikacijske protokole



64

Storitveno vodilo

- Programska arhitektura, implementirana s tehnologijami iz kategorije vmesne infrastrukture (*middleware infrastructure*)
- V osnovi gre za arhitekturo
 - oz. množico pravil in principov integracije večjega števila aplikacij čez infrastrukturo tipa vodilo
- **Temelji na preverjenih standardih**

65

ESB – osnovni namen

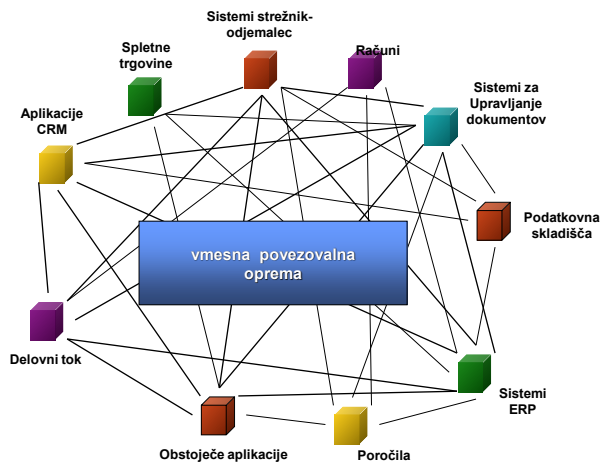
integracija različnih aplikacij in storitev preko skupnega komunikacijskega vodila

- ESB zagotavlja **ŠIBKO SKLOPLJENOST aplikacij in storitev**
 - aplikacije in storitve komunicirajo prek vodila brez neposrednih odvisnosti in zavedanja o drugih sistemih, priklapljenih na vodilo.
- ESB omogoča:
 - usmerjanje, preoblikovanje in obogatitev sporočil,
 - pretvorbo protokolov (npr. HTTP ↔ JMS).
- Koncept ESB izhaja iz potrebe po zamenjavi starih načinov integracije (point-to-point)

66

Pristop: point-to-point

- **Podatkovna povezovalna oprema** omogoča dostop do podatkovnih virov (npr. ODBC, JDBC, ...).
- **Sporočilno orientirana povezovalna oprema** (Message Oriented Middleware).
- **Posredniki zahtev objektov** skrbijo za komunikacijo med porazdeljenimi objekti (npr. CORBA, RMI, ...).
- Mehanizmi za proženje oddaljenih procedur (**RPC, RMI, ...**).
- Vsaka integracija pomeni ločeno povezavo med dvema sistemoma.



67



Težava point-to-point

Sčasoma integracija po principu point-to-point privede do težav pri upravljanju in vzdrževanju povezav.

Pogosto nastanejo »špageti« rešitve z visoko stopnjo medsebojnih odvisnosti.

Posamezne rešitve so odvisne od več podpornih sistemov;

- zamenjava obstoječih podpornih rešitev je zelo težavna ali praktično nemogoča.

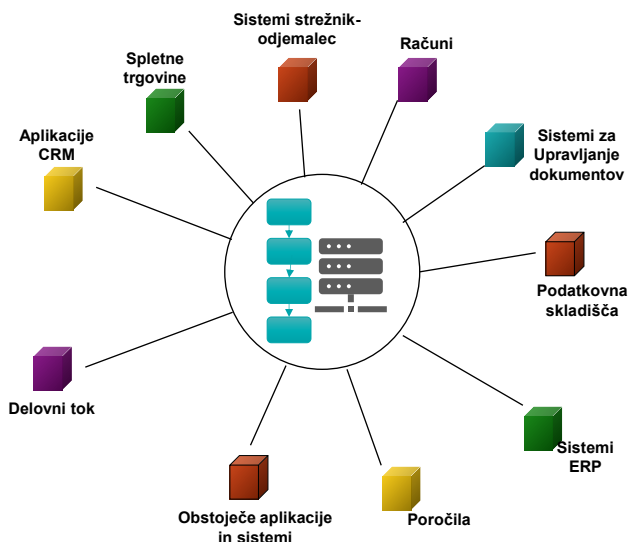
Skaliranje in dodajanje novih sistemov postane drago in počasno.

This Photo by Unknown Author is licensed under CC BY

68

Pristop: skupno vodilo

- Centralizirano usmerjanje in obdelava sporočil prek skupnega vodila.
- Enotna točka integracije za vse aplikacije in storitve.
- Upravljanje izvajanja poslovnih procesov na podlagi definiranih delovnih tokov (workflow/BPM).
- Lažje nadzorovanje, dnevniško beleženje in reševanje napak.



69

Zakaj uporabiti ESB

Večja AGILNOST organizacije

- eden glavnih razlogov, da podjetja uporabijo ESB kot hrbtenico svoje IT infrastrukture

ESB omogoča izpostavitve in ponovno uporabo obstoječih sistemov preko komunikacijskih, transformacijskih in varnostnih mehanizmov.

Zmanjša verjetnost implementacije SOA anti-vzorcev (tesno sklopljene storitve, ad-hoc integracije, podvajanje logike).

70

Kanonični format sporočil v ESB

Podatki potujejo po vodilu v **kanonični (standardizirani)** obliki,

- najpogosteje v obliki XML ali JSON

Kanonični format pomeni, da

- obstaja en konsistenten format sporočil, ki se prenašajo po vodilu,
- vsaka aplikacija, priklopljena na vodilo, lahko z uporabo tega formata komunicira s katerokoli drugo aplikacijo.

Sporočila lahko ESB po potrebi transformira iz/na kanonični format.

71

ESB in SOA

- **POMEMBNO:** storitveno vodilo samo po sebi ne implementira storitveno usmerjene arhitekture (SOA)!
 - ESB pa zagotavlja lastnosti in funkcionalnosti, ki omogočajo implementacijo SOA.
- Ni nujno, da storitveno vodilo temelji na spletnih storitvah
 - lahko uporablja tudi druge protokole in tehnologije (REST, JMS, ...).
- Storitveno vodilo mora
 - temeljiti na **odprtih standardih**
 - biti **prilagodljivo in razširljivo**
 - omogočati implemenentacijo različnih **vzorcev SOA**

72

ESB v kontekstu SOAe



Povezljivost podatkov/adapterji

HTTP(SOAP, REST, XML/JSON),
FTP, SFTP, File,
JMS, ...



Transformacija podatkov

XSLT za XML transformacije,
JSON transformacije,
XPath in XQuery za procesiranje XML sporočil,
skriptni jeziki (npr. JavaScript, Groovy),
...



Inteligentno usmerjanje

usmerjanje na osnovi vsebine sporočil (content-based routing),
napredno usmerjanje na osnovi pravil (rule-based routing).



API po meri

možnost dodajanja lastnih adapterjev ali komponent.

73

ESB v kontekstu SOAe



Koreografija in orkestracija delovnih tokov

vizualno ustvarjanje delovnih tokov in opis zaporedja korakov za pretok podatkov,
avtomatizacija poslovnih procesov.



Storitve upravljanja

orodja za upravljanje namestitev, različic in konfiguracij sistema,
obveščanje o napakah in opozorilih,
zapisovanje dnevnikov (logging, auditing),
nadzor delovanja (monitoring).



Prožilci (triggers)

zmožnost definiranja prožilcev in časovno vodenih akcij,
reakcija na dogodke (event-driven integracija)

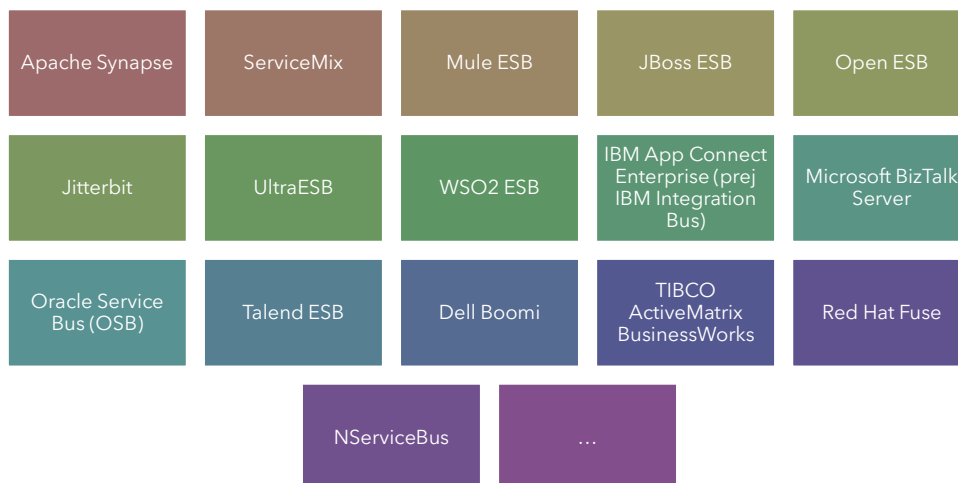


Upravljanje varnosti

avtentikacija, avtorizacija, šifriranje, upravljanje ključev in certifikatov, centralizirane varnostne politike.

74

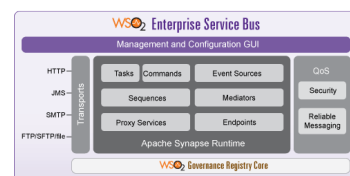
ESB produkti



75

WSO2 ESB

<http://wso2.com/products/enterprise-service-bus/>



- Ponuja vse mehanizme, ki so potrebni v sklopu ESB
 - usmerjanje na osnovi vsebin sporočil,
 - izenačevanje obremenitev (load-balancing),
 - nadomestni načini delovanja (fail-over),
 - omejevanje števila dostopov za določeno časovno obdobje (throttling),
 - preklapljanje protokolov,
 - transformacije podatkov
 - ...
- Nudi podporo za izgradnjo dogodkovno-vodene arhitekture (EDA)
- Prijazen uporabniški vmesnik

76

WSO2 ESB – možnosti povezovanja

- **Prenosni protokoli:** HTTP, HTTPS, POP, IMAP, SMTP, JMS, AMQP, FIX, TCP, UDP, FTPS, SFTP, CIFS, MLLP, SMS
- **Formati in protokoli:** JSON, XML, SOAP 1.1, SOAP 1.2, WS-*, HTML, EDI, HL7, OAGIS, Hessian, Text, JPEG, MP4, Vsi binarni formati, CORBA/IIOP
- **Adapterji za rešitve ERP in druge sisteme:** SAP BAPI & IDoc, PeopleSoft, MS Navision, IBM WebSphere MQ, Oracle AQ, MSMQ
- **Adapterji za storitve v oblakih:** Salesforce, Paypal, LinkedIn, Twitter, JIRA

77

WSO2 ESB – usmerjanje, mediacija in transformacije

- **Usmerjanje sporočil na osnovi**
 - zaglavja sporočil,
 - vsebine sporočil,
 - pravil in
 - prioritet
- **Mediacija**
 - filtriranje sporočil, seznam prejemnikov, zanesljiva dostava,
 - integracija s podatkovnimi bazami
 - objava dogodkov
 - zapisovanje dnevnikov in pregledovanje,
 - validacija
- **Transformacije**
 - XSLT 1.0/2.0, XPath, XQuery, Smooks

78

WSO2 ESB – lahko, razvijalcu prijazno in enostavno okolje

- Deklarativni razvoj s konfiguracijo (namesto kodiranja)
- Enostavna konfiguracija mediacij s toleranco do napak in podporo za upravljanje napak
- Več načinov nameščanja – na privatne strežnike, strežnike v oblakih (WSO2 Enterprise Service Bus-as-a-Service)
- Razširitev konfiguracijskega jezika z lastnim DSL preko predlog
- Vključitev skriptne kode (JavaScript, Jruby, Groovy...) za implementacijo lastnih mediatorjev
- Integriran s SVN, Maven, Ant in drugimi standardnimi orodji za razvoj in nameščanje
- Integriran z WSO2 Developer Studio, ki je IDE za WSO2 produkte

79



Uporabniki WSO2 ESB

<http://wso2.com/casestudies/>

80

WSO2 – eBay case study

<http://wso2.com/download/wso2-ebay-case-study.pdf>



- eBay je ena največja spletnih tržnic
- **eBay uporablja WSO2 ESB za procesiranje več kot 1 milijarde transakcij na dan**
- V podjetju eBay so 6 mesecev vrednotili različne rešitve (programske in strojne) ESB z namenom poiskati najboljšo kombinacijo
 - tako odprtokodne kot komercialne izdelke ESB
- Po procesu vrednotenja so se odločili za 100% odprtokodno rešitev WSO2, ki se je izkazalo za rešitev, ki najbolje ustreza njihovim zahtevam po zmožnosti procesiranja
 - WSO2 storitveno vodilo se je izkazalo kot najbolj učinkovito tako v hitrosti kot zanesljivosti delovanja
 - poleg tega se je WSO2 ESB izkazalo kot edino primerno za kasnejšo nadgradnjo za potrebe podjetja eBay
- Prva namestitev se je izvedla 2009, ki je bila odgovorna za cca 1 milijon klicev na dan
 - V enem letu so se vsi klici preusmerili na WSO2 ESB

81

Življenjski cikli razvoja SOA rešitev

82

Kaj je metodologija razvoja IS?

83

Metodologija razvoja IS

„strukturiran sklop načel, metod, postopkov, standardov in smernic, ki usmerjajo načrtovanje, razvoj, uvedbo in vzdrževanje informacijskega sistema“

Avison, D. E., & Fitzgerald, G. (2006). Information systems development: Methodologies, techniques and tools (4th ed.). McGraw-Hill Education.

- Vključuje pristope in prakse za:
 - načrtovanje, analizo, oblikovanje, implementacijo, testiranje in vzdrževanje programske opreme
- Cilj metodologije je, da razvoj poteka:
 - učinkovito in ponovljivo,
 - v okviru časovnih in finančnih omejitev ter
 - z izpolnjevanjem potreb in pričakovanj uporabnikov.

84

Metodologija razvoja IS

- Zajema vse, kar počnemo, da pridemo do končnega rezultata
- Običajno opredeljuje:
 - **osnovne postopke** razvoja (analiza, načrtovanje, implementacija, testiranje, uvedba, vzdrževanje),
 - **podporne postopke** (vodenje projekta, zagotavljanje kakovosti, obvladovanje sprememb, upravljanje konfiguracije),
 - **načine komunikacije in sodelovanja** med vpletenimi (vloge, odgovornosti, sestanki, artefakti),
 - **pravila odločanja** (kdo, kdaj in na kakšni osnovi sprejema ključne odločitve),
 - **podporna orodja** (repozitoriji kode, orodja za vodenje zahtev, testna orodja, CI/CD),
 - in druge elemente, ki zagotavljajo urejen in sledljiv razvoj.

85

Zakaj potrebujemo metodologijo razvoja IS?

86

Potreba po metodologiji razvoja IS

- Serija raziskav CHAOS (Standish Group) od 1994 dalje kaže presenetljivo nizko uspešnost IT projektov.
- V CHAOS 2009 (≈ 50.000 projektov) so ugotovili:
 - 32 % uspešnih projektov
 - zaključeni pravočasno
 - znotraj proračuna
 - z zahtevanimi funkcionalnostmi
 - 44 % "challenged" projektov
 - zamude
 - prekoračeni stroški
 - okrnjene funkcionalnosti
 - 24 % neuspešnih projektov
 - projekt odpovedan
 - ali sicer zaključen, a se nikoli ne uporablja

Table 1
Standish project benchmarks over the years

Year	Successful (%)	Challenged (%)	Failed (%)
1994	16	53	31
1996	27	33	40
1998	26	46	28
2000	28	49	23
2004	29	53	18
2006	35	46	19
2009	32	44	24

Vir: <http://www.few.vu.nl/~x/chaos/chaos.pdf>

87

Potreba po metodologiji razvoja IS danes

- CHAOS 2020 (zadnja izdaja poročila) pokaže, da je slika še vedno podobna:
 - 31 % projektov uspešnih,
 - 50 % "challenged",
 - 19 % neuspešnih.
- Pri primerjavi metodologij poročilo ugotavlja, da:
 - projekti, ki uporabljajo agilne pristope, so približno 3-krat bolj verjetno uspešni kot projekti z Waterfall pristopom;
 - Waterfall projekti imajo približno 2-krat večjo verjetnost neuspeha.
- Priporočilo: premik od razumevanja razvoja programske opreme kot enkratnega projekta k stalnemu izboljševanju in kontinuiranemu razvoju.



<https://www.successthroughsafe.com/blog-1/2021/11/13/standish-chaos-report-2021>

88

Kaj nam poročila povedo o razvoju IS?

- Uspešnost projektov se že več desetletij giblje okoli ~30 % - ne glede na tehnološki napredek.
 - Večina projektov ima težave s časom, stroški in obsegom (»challenged«).
- Uporaba primerne metodologije (agilna, hibridni pristop) statistično izboljša rezultate, klasični Waterfall pa ostaja bolj tvegan.
- Ključni dejavniki uspeha so:
 - ljudje (ekipa, sponzor, kultura sodelovanja),
 - proces (metodologija, iterativnost, majhni koraki),
 - organizacija (podpora vodstva, jasni cilji, "good place").

89

Modeli razvoja IS

Skozi zgodovino so se razvili in izpopolnili različni modeli življenjskega cikla razvoja IS, ki opisujejo, kako si faze razvoja časovno sledijo in kako se ponavljajo

V praksi se modeli pogosto kombinirajo in prilagodijo značilnostim projekta in organizacije

zaporedni (»vodopadni«) model,

iterativni model,

inkrementalni model,

spiralni model,

agilni modeli (npr. Scrum, Kanban),

.....

90

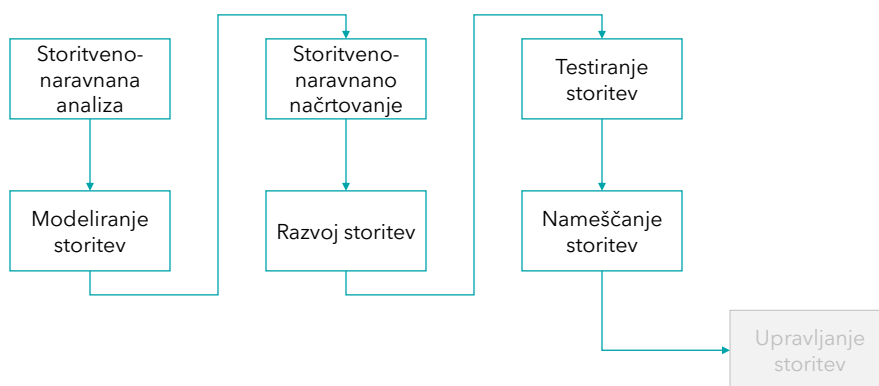
Pristopi k razvoju SOA rešitev

- Razvoj SOA rešitve, tako kot vsak drug projekt razvoja programske opreme, zahteva izbiro ustreznega pristopa.
- Pri storitveno usmerjeni arhitekturi običajno razlikujemo tri splošne pristope:
 - BOTTOM-UP pristop**
 - TOP-DOWN pristop**
 - AGILNI (»middle-out«) pristop**
(srečanje nekje na sredini med top-down in bottom-up)

V vsakem pristopu je zahtevana določena stopnja **storitveno naravnane analize in načrtovanja vstoritev in inventarja storitev**

91

SOA: Osnovne faze življenjskega cikla



Po zaključku osnovnih faz sledi faza upravljanja storitev

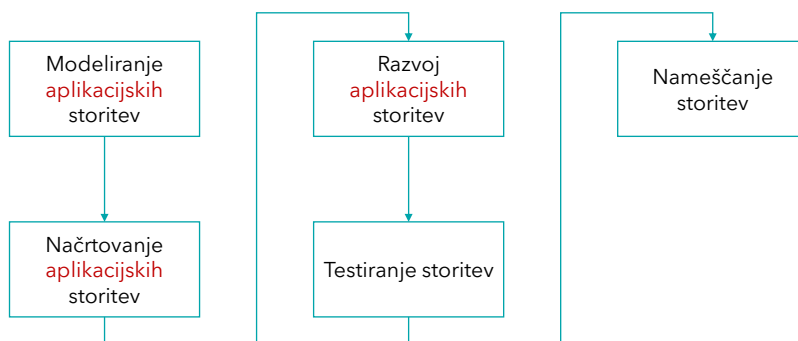
92

BOTTOM-UP pristop

- V praksi je najpogostejši pristop pri uvajanju SOA
- Tipični razlogi:
 - spletne storitve uporabimo, da obstoječe aplikacije in njihove funkcionalnosti izpostavimo v obliki storitev,
 - na voljo orodja, platforme in ogrodja za hiter razvoj aplikacijskih spletnih storitev,
 - v organizacijah pogosto primanjkuje znanja s področja storitveno usmerjenega načrtovanja, zato se izhaja iz obstoječe logike,
 - pristop omogoča postopno uvajanje – brez večjih začetnih sprememb arhitekture.

93

BOTTOM-UP pristop – kako poteka



- Storitve se gradijo na podlagi trenutnih **aplikacijskih zahtev in so tipično zasnovane tako, da:**
 - ovijejo obstoječo aplikacijsko logiko (angl. wrapping),
 - izpostavijo izbrane funkcionalnosti obstoječih sistemov,
 - omogočijo ponovno uporabo teh funkcionalnosti v drugih rešitvah.

94

Prednosti BOTTOM-UP pristopa

Ponovna uporaba
obstoječih
komponent

obstoječa logika se
ovije v storitve, zato
ni potrebno vsega
razviti na novo.

Hiter začetek in
hiter razvoj

možno je
razmeroma hitro
pokazati prve
rezultate.

Nižji začetni
stroški

manjša začetna
investicija v analizo
in preoblikovanje
celotne arhitekture.

95

Slabosti BOTTOM-UP pristopa

Osnovna arhitektura
ostaja
nespremenjena

pristop sam po sebi
ne zagotavlja
nastanka resnično
storitveno usmerjene
arhitekture.

Redko pride do
prave storitvene
naravnosti

storitve so pogosto
preblizu obstoječim
aplikacijam in niso
dovolj poslovno
usmerjene.

Pomanjkanje
centraliziranega
načrtovanja

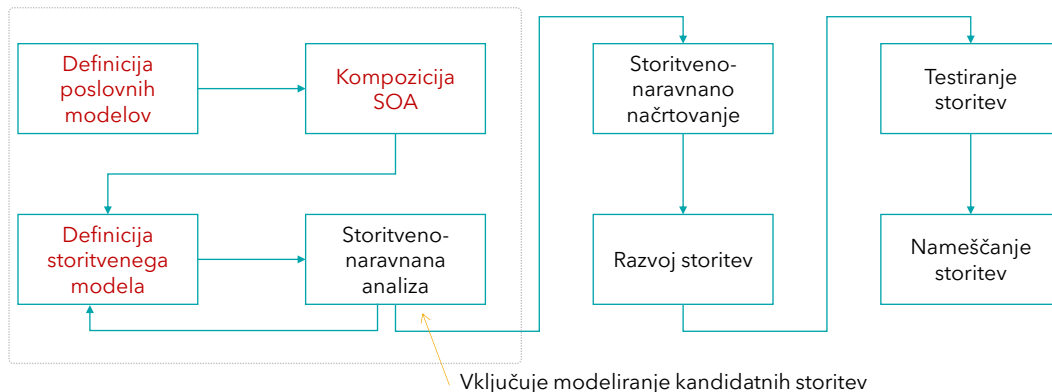
brez celovitega
načrta obstaja
tveganje neskladja
med storitvami in
celotno arhitekturo,

storitve se lahko
podvajajo, nastajajo
redundance in
povečuje se
kompleksnost.

Posledica so višji
stroški vzdrževanja
in nadgrajenij na
daljši rok.

96

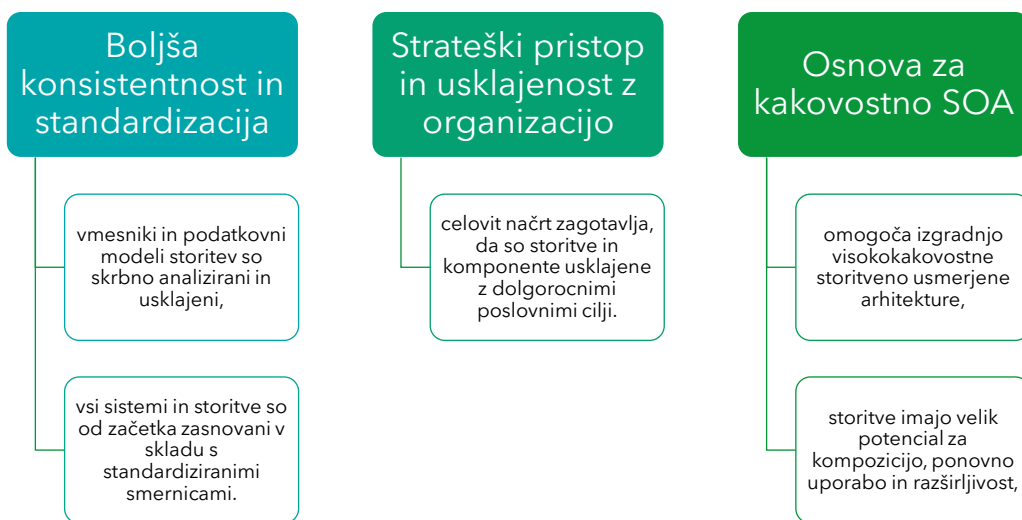
TOP-DOWN pristop



- Glavni poudarek je na **temeljiti začetni analizi in poslovnem načrtovanju**.
- **Omogoča razvoj storitveno usmerjenega poslovnega modela** za celotno organizacijo:
 - iz poslovnih ciljev in procesov izpeljemo poslovne storitve,
 - vse ključne poslovne funkcionalnosti se načrtno izpostavijo v obliki storitev,
 - nato se išče najprimernejša tehnična realizacija teh storitev.

97

TOP-DOWN - Prednosti



98

TOP-DOWN - Slabosti

Višji začetni stroški in dolgotrajno načrtovanje

zahteva temeljito analizo in načrtovanje, preden je mogoča večja implementacija,

pogosto je potrebno več investicije vnaprej.

Dolgotrajna pot do prvih rezultatov

traja lahko precej časa, preden so uporabnikom na voljo prve storitve,

obstaja tveganje, da del zasnove ob implementaciji že ni več povsem aktualen.

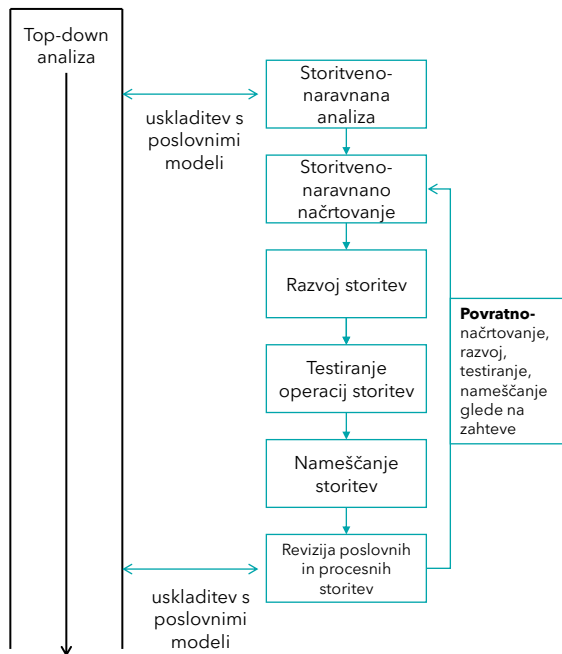
Manj takojšnjih koristi

težje je pokazati hitre, vidne rezultate, kar lahko oteži podporo vodstva.

99

Agilni pristop

- **Kombinacija** top-down in bottom-up pristopa.
- Vpeljan je **dodatni proces analize** poslovnih zahtev, ki poteka:
 - sočasno z načrtovanjem storitev,
 - in iterativnim razvojem ter uvajanjem storitev.
- Namesto velikega enkratnega načrta se arhitektura gradi **po korakih**
 - pri čemer se sproti upošteva povratne informacije in spreminjajoče se zahteve.
- Gre za **kompleksnejši pristop**, ker zahteva dobro usklajevanje med:
 - poslovno analizo,
 - arhitekturo
 - in razvojnimi ekipami.



100

Agilni pristop - Prednosti

Prilagodljivost in hitra odzivnost

sprotno prilagajanje glede na povratne informacije uporabnikov in poslovnega okolja.

Postopen razvoj z osredotočenostjo na prioritete

storitve se uvajajo iterativno, najprej tiste z največjo poslovno vrednostjo.

Boljša komunikacija med ekipami

pogoste iteracije, kratki cikli in redni usklajevalni sestanki spodbujajo

sodelovanje med poslovnimi in tehničnimi ekipami.

Hitrejša vidnost rezultatov

uporabniki prej dobijo prve uporabne storitve, ki jih je mogoče takoj preizkusiti.

101

Agilni pristop - slabosti

Nevarnost izgube širše arhitekturne slike

zaradi osredotočenosti na kratke iteracije lahko primanjkuje celovitega arhitekturnega načrta, kar lahko vodi v neskladnosti in neenotne vzorce.

Upravljanje kompleksnosti

z naraščajočim številom storitev postane zahtevno zagotavljati enotnost, standarde in ponovno uporabo.

Tveganje "nikoli končanega" projekta

če se zahteve ves čas spreminjajo in prioritizirajo, obstaja tveganje, da projekt nima jasnega konca, oz. da nikoli ne doseže stabilne različice.

102

Storitveno-naravnana analiza

- V tej začetni fazi se določijo cilji SOA rešitve (vizija)
- Definirajo se različni nivoji storitev
- Vključuje tudi aktivnost analize inventarja storitev



103

Storitveno naravnana analiza - **ključni vprašanja:**

- **KATERE STORITVE** moramo zgraditi?
- **KATERI DEL LOGIKE** mora biti zajet v posameznih storitvah?
- Stopnja truda, ki se vloži v fazo storitveno-naravnane analize se odraža na tem, kako kompletno lahko podamo odgovore na ta vprašanja

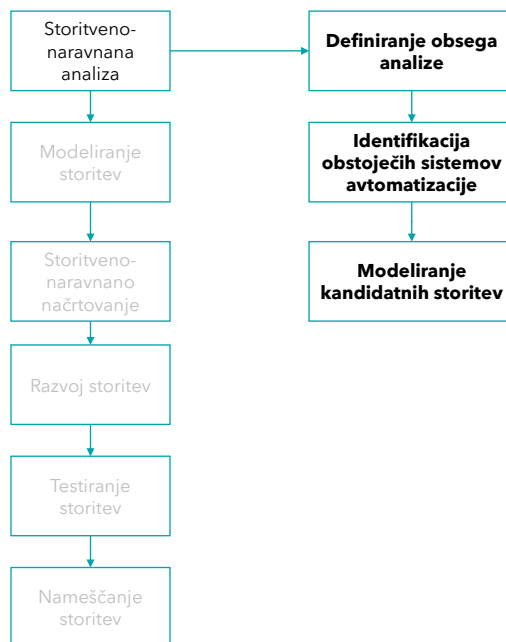
104

Cilji storitveno naravnane analize

- Definicija **začetne množice kandidatnih operacij** storitev
- **Grupiranje kandidatnih operacij** storitev v logične kontekste – **oblikovanje kandidatnih storitev**
- Identifikacija ponovno uporabnih storitev
- Identifikacija morebitnih izzivov z zagotavljanjem avtonomnosti posameznih storitev
- Definicija modelov kompozicij storitev

105

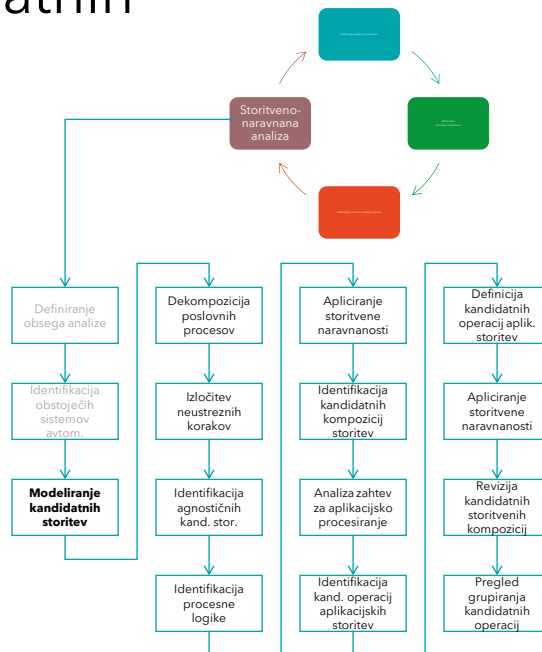
Proces storitveno naravnane analize



106

Modeliranje kandidatnih storitev

- Korak v fazi analize - **konceptualizacija storitev in njihovih zmožnosti**
- Izvede se pred dejanskim načrtovanjem in razvojem storitev
- Rezultat: **množica kandidatnih storitev**



107

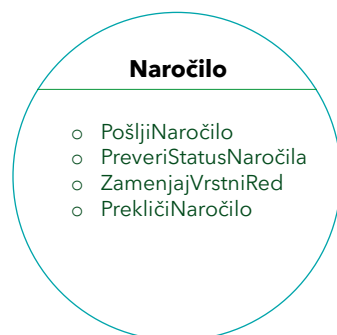
Storitve : kandidatne storitve

- Osnovni cilj storitveno-naravnane analize je **definicija storitev na logičnem nivoju**
 - Da se ve, kaj se bo načrtovalo in zgradilo.
 - Ne izvaja se še načrtovanje!
 - V fazi analize se pripravi predlog (SZPO), ki je vhod v fazo načrtovanja.
- Z drugimi besedami:
 - **faza analize pripravi množico abstraktnih definicij storitev**, ki bodo (ali ne bodo) realizirane skozi fazo konkretnega načrtovanja in implementacije storitev

108

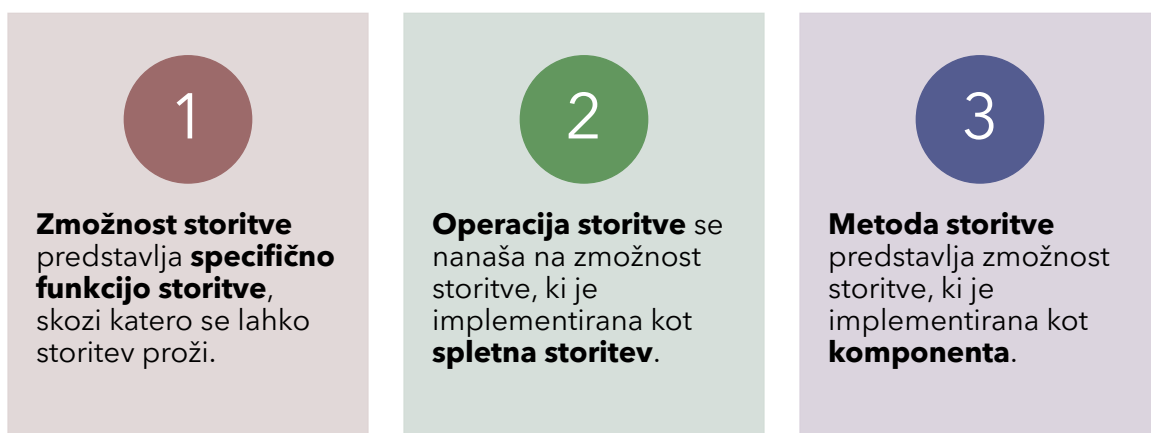
Zmožnost storitve

- Vsaka storitev je načrtovana glede na svoj **funkcijski kontekst** - predstavlja množico funkcij, povezanih s tem kontekstom
- Dokler se ne ve, kako bo storitev zgrajena, se funkcije storitve imenujejo **zmožnosti oz. sposobnosti storitve**
- Storitev ima lahko **eno ali več** zmožnosti (funkcij)



109

zmožnost : operacija : metoda



Izraz "zmožnost" se je uvedel zato, da se jasno loči kdaj govorimo o storitvi kot abstraktnem pojmu in kdaj o implementaciji storitve

110

Modeli storitev

- Storitve lahko razdelimo glede na
 - tip logike, ki jo ovijejo
 - sposobnost ponovne uporabe, in
 - kako se logika navezuje na obstoječe domene v podjetju

111

Modeli storitev

- Glede na prej omenjene načine delitve so se definirale tri splošne klasifikacije, ki predstavljajo **osnovni storitveni model**:
 - **entitetne storitve** (entity services)
 - **poslovne storitve** (task services)
 - **podporne storitve** (utility services)

112

Entitetna storitev

- Entitetne storitve so povezane z osnovnimi poslovnimi objekti ali entitetami (npr. stranke, izdelki, naročila).
 - V vsakem podjetju se najdejo poslovni dokumenti, ki definirajo in vsebujejo poslovne entitete, relevantne za organizacijo
- Te storitve zagotavljajo dostop in manipulacijo podatkov, ki so povezani s temi entitetami, ter omogočajo operacije, kot so pridobivanje, shranjevanje, posodabljanje in brisanje podatkov.
- Značilnost entitetnih storitev je **visok nivo ponovne uporabe**
 - Uporabi se lahko v več poslovnih procesih
- Entitetne storitve se imenujejo tudi poslovne entitetne storitve

Zaposleni

```
GetTedenskiLimitUr
UpdateTedenskiLimitUr
GetZgodovina
UpdateZgodovina
DeleteZgodovina
DodajProfil
GetProfil
UpdateProfil
DeleteProfil
```

Primer entitetne storitve. Večina zmožnosti je povezanih s klasičnimi CRUD (create, read, update, delete) metodami.

113

Poslovna storitev

- Poslovne storitve izvajajo specifične poslovne naloge ali procese, kot so obdelava naročila, odobritev posojila ali generiranje poročil.
 - Te storitve tipično vključujejo različne entitete, da dosežejo določen poslovni cilj.
- Poslovna storitev je **vezana na specifično poslovno opravilo ali poslovni proces**
- Ta tip storitve ima **najnižjo stopnjo ponovne uporabe**
- Pogosto je tovrstna storitev postavljena v vlogo kontrolnika kompozicije, ki je odgovoren za sestavljanje drugih storitev

Naročilo

Pošlji
...

Primer poslovne storitve z eno izpostavljenostjo, ki zahteva začetek izvajanja poslovnega procesa, ki ga storitev ovija oz. izpostavlja.

114

Podporna storitev

- Podporne storitve zagotavljajo splošne funkcionalnosti, ki jih uporabljajo druge storitve, neodvisno od konkretnih poslovnih procesov ali entitet.
- V avtomatizaciji poslovanja pogosto potrebujemo tudi storitve z zmožnostmi, ki niso **poslovne narave**
 - ne izvajajo poslovnih procesov,
 - nimajo povezave s poslovnimi entitetami
- Primer storitve prikazuje slika, kjer storitev omogoča zmožnosti preoblikovanja, ki se lahko proži v različnih poslovnih procesih
 - Predstavljajo visok nivo ponovne uporabnosti
- Podporne storitve imenujemo **tudi aplikacijske storitve, infrastrukturne storitve ali tehnološke storitve**

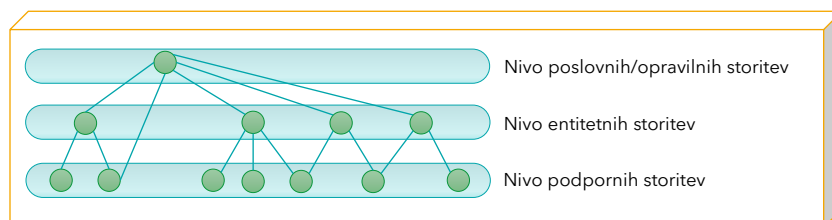
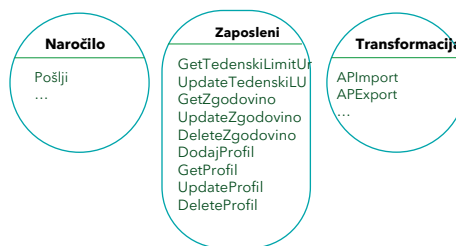


Primer podporne storitve, ki predstavlja množico zmožnosti, povezanih s transformacijami podatkov, ki se izvajajo med procesiranjem sporočil v procesu.

115

Modeli storitev in nivoji storitev

- Znotraj inventarja se storitve klasificirajo z uporabo modelov storitev, pri čemer se organizirajo v logične nivoje storitev



116

Storitve – izzivi

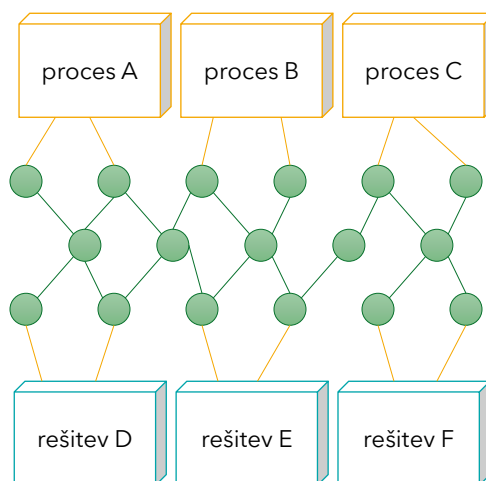
Najpogostejši izzivi storitvenega načrtovanja:

- načrtovanje pogodb
- agnostičen : ne-agnostičen pristop k načrtovanju
- enkapsulacija podedovanih aplikacij

117

Agnostične storitve (ponovimo)

- Osnovni cilj storitvene naravnosti je izgradnja storitev, pri čemer je **večina funkcijsko agnostičnih**
 - da storitve nudijo splošne funkcionalnosti, ki so uporabne v različnih procesih.



118

Agnostične : ne-agnostične storitve

- Beseda „agnostic“ izhaja iz grške besede „agnostos“, ki v prevodu pomeni „brez znanja“
- Logika, ki je dovolj oz. tako splošna, da **ni vezana na** (nima znanja o) **določeno nalogo** (korak v procesu), je klasificirana kot **agnostična logika**
 - Ker se znanje, vezano o opraviu namenoma izloči, je agnostična logika več-namenska in ponovno uporabljiva v različnih procesih
- Na drugi strani se logika, ki **je vezana na** (ima znanje o) **določeno-specifično nalogo**, označuje kot **ne-agnostična logika**
- Funkcijsko agnostične storitve so zasnovane tako, da so univerzalne in se lahko ponovno uporabljajo v različnih kontekstih.

119

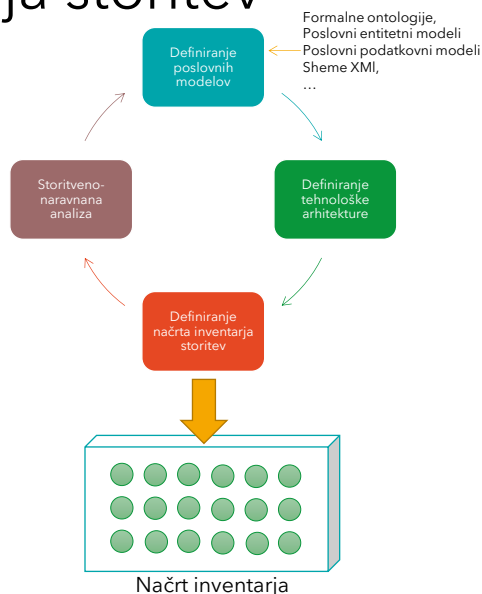
Inventar storitev

- je **neodvisno standardizirana in upravljana množica komplementarnih storitev znotraj meje podjetja**
 - Najboljša praksa je, če so vse storitve dostopne preko istega inventarja storitev
 - V večjih okoljih se lahko zgodi, da standardizacija čez celotno podjetje ni možna. V tem primeru se oblikujejo **domenski inventarji storitev**.
- Načrti inventarjev storitev so običajno definirani vnaprej - v fazi analize
 - V fazi analize se osredotočimo predvsem na izogibanje prekrivanja med posameznimi storitvami → **normalizacija storitev**

120

Cikel analize inventarja storitev

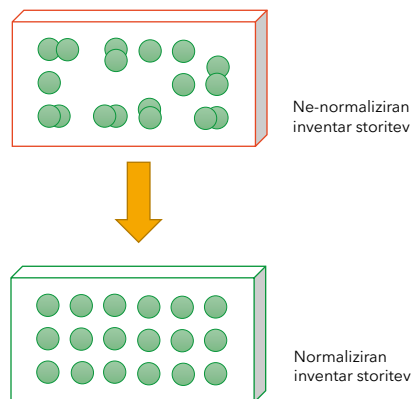
- Del top-down pristopa k razvoju storitev
- Iterativno-inkrementalni pristop
- Načrt inventarja storitev je rezultat ponavljajočih iteracij korakov, ki vključujejo storitveno-naravnano analizo



121

Normaliziran inventar storitev

- Inventar storitev, katerih funkcionalnosti se prekrivajo predstavlja ne-normaliziran inventar
- Storitve se lahko skupno modelirajo tako, da je **vsaka storitev v okviru svojih meja in se ne prekriva z drugimi storitvami**
 - Rezultat je **inventar storitev z visoko stopnjo funkcijske normalizacije**



122

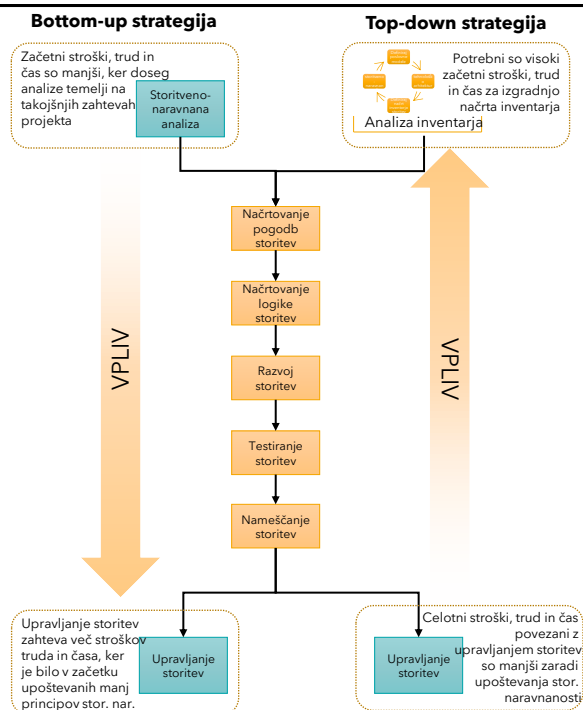
Upravljanje inventarja storitev

- Upravljanje SOA vključuje več aktivnosti
- Med drugim tudi **upravljanje inventarja storitev**
- Izbran pristop k izgradnji SOA rešitve se odraža v trudu, ki ga je potrebno vložiti v fazi upravljanja inventarja storitev
 - **Top-down** pristop k izgradnji SOA rešitve **zmanjša trud**, ki je potreben za upravljanje inventarja storitev
 - **Bottom-up** pristop **poveča stopnjo truda** v fazi upravljanja inventarja storitev

123

Upravljanje inventarja storitev

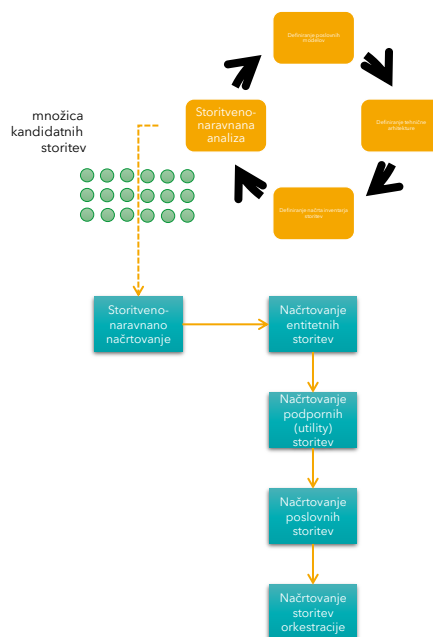
- Vsaka strategija izgradnje SOA rešitev ima svoje prednosti
- Pri izbiri strategije je potrebno upoštevati tudi trud, povezan z upravljanjem storitev



124

Storitveno-naravnano načrtovanje

- Je proces, ki se nadaljuje tam, kjer se je zaključila storitveno-naravnana analiza
- **Vhod** v proces storitveno-naravnane načrtovanja so **kandidatne storitve**
- **Za vsak model storitve** se izvede **ločen proces storitveno-naravnane načrtovanja**



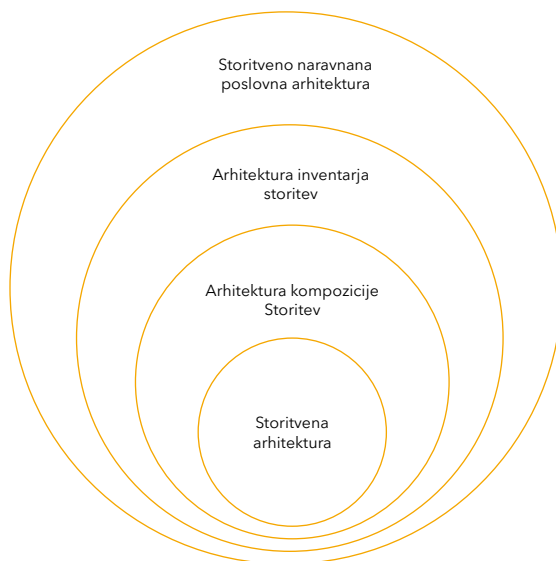
125

Tipi storitveno usmerjenih arhitektur

126

Tipi storitveno-usmerjenih arhitektur

Vsak od naštetih tipov SOA tipov ima svoj doseg



127

Tipi storitveno-usmerjenih arhitektur

- V okviru storitveno usmerjene arhitekture razlikujemo več nivojev:
 - **Storitvena arhitektura**
 - notranja zgradba posamezne storitve (kako je storitev implementirana).
 - **Arhitektura kompozicije storitev**
 - kako posamezne storitve povežemo v **večje poslovne rešitve in procese**.
 - **Arhitektura inventarja storitev**
 - celovit pregled in organizacija **vseh storitev v podjetju**.
 - **Storitveno naravnana poslovna arhitektura**
 - najvišji nivo arhitekture, kjer so **poslovne sposobnosti in procesi opisani kot storitve**.

128

Arhitektura storitve

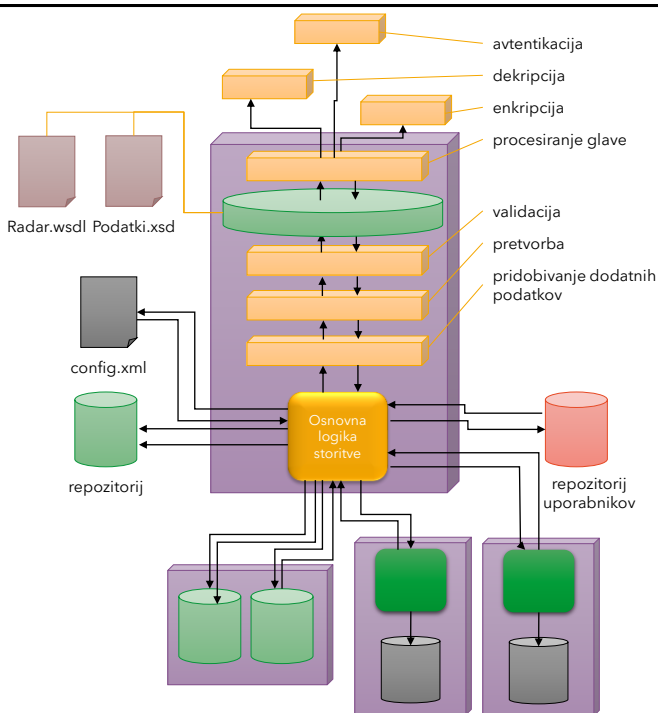
je **arhitektura posamezne storitve, omejena na njen fizični načrt in implementacijo.**

- Lahko jo primerjamo s **komponento** v komponentni arhitekturi:
 - storitev ima jasen vmesnik in notranjo zgradbo,
 - za implementacijo ene storitve je lahko uporabljenih več komponent, podatkovnih virov ali modulov.

129

Arhitektura storitve - zahteve

- Za arhitekture storitev, zlasti za **agnostične storitve**, velja, da morajo storitve delovati kot:
 - **neodvisne** - čim manj neposrednih odvisnosti od konkretnih aplikacij,
 - **samo-zadostne** - vsebujejo vso potrebno logiko za izvajanje svoje funkcionalnosti,
 - **samo-vsebujoče aplikacijske rešitve** - storitev je mogoče uporabiti brez poznavanja njene notranje implementacije,
 - z **jasno definiranimi vmesniki** in dogovorjenimi podatkovnimi formati.

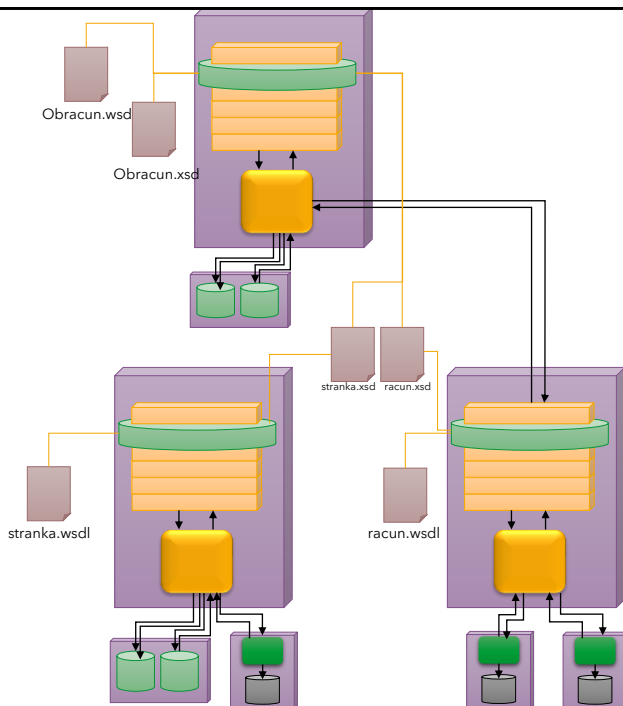


130

Arhitektura kompozicije storitev

opisuje, **kako posamezne storitve povežemo v večje poslovne rešitve:**

- procesne tokove (orkestracije),
 - poslovne delovne tokove,
 - sestavljene (kompozitne) storitve.
- Arhitekturo kompozicije lahko primerjamo z **integracijsko arhitekturo**, še posebej v primerih, ko storitve:
 - "ovijajo" in izpostavijo funkcionalnosti **ločnih podedovanih (legacy) sistemov**,
 - delujejo kot **posrednik** med različnimi aplikacijami in podatkovnimi viri.



131

Arhitektura inventarja storitev

- Predstavlja **celovit nabor storitev** v določenem okolju (podjetju, poslovni domeni).
- Opredeli:
 - **kategorije in domene storitev**,
 - pravila za **poimenovanje, verzioniranje** in **ponovno uporabo**,
 - odnose med storitvami in poslovnimi zmožnostmi.
- Cilj je, da storitve niso množica ad-hoc rešitev, ampak **urejen "inventar"**, ki ga je mogoče dolgoročno vzdrževati in širiti.

132

Storitveno-naravnana poslovna arhitektura

- Arhitektura na najvišjem poslovnem nivoju, ki zajema vse ostale arhitekture.
 - Poslovne zmožnosti, procesi in organizacijske enote so opisani in povezani kot storitve.
- Arhitektura določa:
 - okvirje in pravila za načrtovanje inventarja storitev,
 - smernice, kako storitve podpirajo poslovne cilje in strategijo organizacije.
- Pravila in usmeritve, določene na tem nivoju, se **prenašajo navzdol** na:
 - arhitekturo inventarja storitev,
 - arhitekturo kompozicij,
 - arhitekturo posameznih storitev.